

Fly GACA — لوحة المدرّب — مواصفة التصميم

القسم operations-it-06 النوع spec الحالة active المالك Founder آخر تحديث 19-08-2026

الحالة: مبنية جزئيًا · يرتبط بـ: خطة العمل 5.2§، `roadmap.md` المرحلة الخامسة أُعيد في: 25-05-2026 · أُعيد تأسيسها على منظومة 2026-08-19 Cloud Run / Postgres

هذه مواصفة بناء لوحة المدرّب/المدرسة بين الشركات (B2B). وهي عمدة قائمة على المواصفة أولًا: فالجزء الصعب ليس واجهة المستخدم، بل نموذج بيانات متعدّد المستأجرين يتيح لمستخدم أن يرى تقدّم آخر دون إضعاف العزل الصارم لكل مستخدم القائم على المنصة اليوم. وهذا يجب أن يُصمّم لا أن يُستعجل.

تُسوّق صفحة `/schools` هذا أصلًا — "لوحة مشرف المدرسة ... جاهزية الفوج للاختبار وتقدّم كل طالب على حدة ... مُتضمّنة مع كل عقد" وتبني هذه المواصفة ذلك الوعد.

ما هو منشور فعليًا. النصف الخاص بمالك المنظمة حي: إذ تعرض `/business/admin` تقرير جاهزية الفوج من `GET /api/org/:orgId/cohort-readiness`، مسنودًا بجدولَي `orgs` و `org_seats` ويرفع `study_progress` من `src/lib/services/studyProgressSync.ts`. أمّا غير المبني فهو دور المدرّب، والتعمّق لكل طالب، وتعيين مسارات القراءة، و — والأهم — بوابة الموافقة في 3§. اقرأ بقية هذه المواصفة بوصفها الفارق.

1. القيد الجوهري

النموذج اليوم عزل صارم لكل مستخدم، ومنذ النقل إلى Cloud Run صار مفروضًا *بنبويّة* لا تعريفياً: فالمتصفّح لا يملك أي وصول إلى قاعدة البيانات إطلاقًا. إذ لا يخاطب سوى `/api/*`، ويقرأ `routes/account.ts` ويكتب صفوفًا لـ `requireUser(req)` ولا أحد سواه، ولا وجود لأي مسار يعيد ملفًا شخصيًا أو سجلّ طيران أو سجلّات لمستخدم آخر.

تتطلب لوحة المدرّب قراءات عابرة بين المستخدمين مُتحكّمًا بها — مدرّب يرى تقدّم طالب. ويقوم التصميم الكامل أدناه على فعل ذلك بأمان.

قرار التصميم المحوري الذي يحلّه: القراءة العابرة بين المستخدمين هي عملية ربط من جانب الخادم تعيد إسقاطًا، لا صفوفًا أبدًا. فتربط واجهة البرمجة `org_seats → users → entitlements → study_progress` في استعلام واحد ولا تعيد سوى حقول الجاهزية المشتقة (نسبة التغطية، آخر درجة تجريبية، جاهز/غير جاهز، `progressUpdatedAt`). ولا يسلم أي مسار مدرّبًا صفّ `profiles` أو `flights` أو `pilot_records` الخاص بطالب، ويُحسب الإسقاط في `org-core.ts` حيث يُختبر وحدويًا. فضمانة العزل خاصيّة في سطح المسارات، لا في ملف قواعد.

ويحلّ هذا محلّ التصميم الأقدم القائل بأن "يكتب عميل كل طالب نسخة محصورة النطاق في الشجرة الفرعية للمدرسة"، وهو تصميم لم يوجد إلّا لأن Firestore لم يكن قادرًا على الربط من جانب الخادم ولا كانت قواعده تعبّر عن إسقاط على مستوى العمود. أمّا Postgres فيفعل الاثنين، ومن ثمّ زالت الكتابة المكرّرة — وما كانت تدعو إليه من تقادم، للمراجعين: هذا أكبر تغيير آليّ منفرد في هذه المواصفة.

2. الأدوار

الدور	من	القدرة
الطالب / الطيار	موجود اليوم — صف في <code>users</code>	يملك بياناته؛ ويختار الانضمام إلى فوج
مالك المنظمة	موجود اليوم — <code>orgs.owner_user_id</code>	يشترى المقاعد، ويهيئ القائمة، ويقرأ ملخص الفوج
المدرّب	جديد	يقرأ تقدّم فوجه؛ ويعيّن مسارات قراءة
مشرف المدرسة	جديد	يدير المقاعد والمدرّبين والقائمة؛ ويرى ملخص الفوج

الطالب على خطة مدرسة موسوم أصلًا بطريقتين: مقعده صف في `org_seats (org_id, email)` — يُعنون بالبريد الإلكتروني، لأن المقاعد تُهيئ قبل أن يكون للمدعو حساب وتُطالب عند أول تسجيل دخول موثّق — وقيمة `entitlements.source` لديه هي `'school'`.

واليوم لا يملك `orgs` سوى فاعل واحد ذي صلاحية، هو `owner_user_id`، ويتحقّق منه `routes/org.ts` عبر `ownedOrg(orgId, uid)`. أمّا المدرّب والمشرّف بوصفهما دورين *متمايزين* فغير موجودين بعد؛ وتضيفهما 4§.

3. الموافقة والخصوصية — من الدرجة الأولى

سجّل الطالب وبيانات دراسته بيانات شخصية. وتقييم أثر حماية البيانات (DPIA) بموجب PDPL بوابة إطلاق أصلًا (`roadmap.md`)؛ ويجب أن تكون الرؤية العابرة بين المستخدمين ضمن نطاق ذلك التقييم.

قاعدتان تلتزم بهما هذه المواصفة:

1. النطاق. يرى المدرّبون بيانات الدراسة وجاهزية الاختبار فقط — تقدّم المدرسة الأفضلية، ودرجات الاختبارات التجريبية، وأعلام الحداثة. ولا يرون السجّل الشخصي. وهذه صحيحة أصلًا بحكم البناء؛ إذ لا يحمل `study_progress.summary` سوى الدرجات ونسب الإكمال — ولا يحمل الإجابات أبدًا — ولا يربط أي مسار جدول `flights` بتقرير الفوج.

2. موافقة صريحة قابلة للإلغاء. عند تسجيل طالب في فوج يرى شاشة موافقة تسمّي بدقّة ما ستراه المدرسة، ويجب أن يقبل. ويمكنه إلغاؤها من الإعدادات، فيقلب ذلك مقعده إلى `consent = false` ويفرّغ حقول الجاهزية التي يعيدها التقرير عنه.

[WARNING!] القاعدة الثانية غير منفذة. فلا يحمل `org_seats` عمود موافقة، ويعيد `GET` `/api/org/:orgId/cohort-readiness` حالياً صف جاهزية عن كل مقعد في القائمة. وسدّ هذا هو أول بند في 8§ وشرط مسبق لتقييم DPIA — وينبغي إنزاله قبل بيع اللوحة لمدرسة ثانية.

4. نموذج البيانات (Postgres)

لا يتغيّر شيء في جداول الطالب نفسه. ويُضاف عمودان وجدول واحد؛ أمّا جدول التقدّم الذي كانت المواصفة القديمة تسمّيه "المرحلة A" فهو موجود أصلاً.

```
-- SHIPPED – the cadet's own progress projection, uploaded by their client
-- (studyProgressSync.ts, upload-only, best-effort). Scores + completion only.
-- CREATE TABLE study_progress (
--   user_id      uuid PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
--   summary      jsonb NOT NULL DEFAULT '{}':jsonb,
--   updated_at   timestampz NOT NULL DEFAULT now()
-- );

-- SHIPPED – the cohort and its roster.
-- orgs          (id, name, owner_user_id, seat_limit, created_at)
-- org_seats     (org_id, email, status, source, expires_at, claimed_by, created_at)

-- NEW – consent, per seat. Written only by the cadet's own consent/revoke route.
ALTER TABLE org_seats
  ADD COLUMN consent      boolean NOT NULL DEFAULT false,
  ADD COLUMN consent_at   timestampz;

-- NEW – staff of an org, beyond the single owner.
CREATE TABLE org_members (
  org_id      uuid NOT NULL REFERENCES orgs(id) ON DELETE CASCADE,
  user_id     uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  role        text NOT NULL CHECK (role IN ('admin','instructor')),
  created_at  timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (org_id, user_id)
);
CREATE INDEX org_members_user_idx ON org_members (user_id);

-- NEW – instructor → cadet reading-path assignment.
CREATE TABLE org_assignments (
  org_id      uuid NOT NULL REFERENCES orgs(id) ON DELETE CASCADE,
  email       citext NOT NULL,
  path_ids    text[] NOT NULL DEFAULT '{}',
  assigned_by uuid REFERENCES users(id),
  updated_at  timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (org_id, email)
);
```

ولا حاجة إلى اسم الطالب وبريده بصورة غير معيارية في القائمة — فالتقرير يربط `users` على `org_seats.email` ويقرأ `profiles` للاسم المعروض في الاستعلام نفسه.

وتبقى محاسبة المقاعد حيث هي: إذ تعدّ `checkSeatLimit` في `org-core.ts` صفوف `org_seats` مقابل `orgs.seat_limit` قبل التهيئة، فلا يمكن لشراء فوج أن يتجاوز حدّ المقاعد. وتحدّ `seatExpiry` المقعد بنافذة دفعة واحدة، وهو ما يمنع فوجًا مدته 90 يومًا من سكّ حسابات `school` دائمة.

5. نموذج الوصول — كيف يُفرض فعليًا

لا ملف قواعد، ولا محاك، ولا جلسة قاعدة بيانات من جانب العميل. فكل قراءة مسار، والمسار هو الحدّ. وهذه ترقية مباشرة على تصميم Firestore، الذي كانت 5s لديه كلها مسوّدة قواعد يلزم اختبارها هجوميًا في محاك. البوابة لكل مسار فوج:

```
// server/src/routes/org.ts – the shipped owner check, generalised
async function orgStaff(orgId: string, uid: string): Promise<OrgRole> {
  // owner, or a row in org_members. Anything else → 403.
}
```

قواعد تلزم بها هذه المواصفة، وكُلّ منها قابل للاختبار كاختبار تكامل اعتيادي:

القاعدة	أين تعيش
لا يستدعي أي مسار <code>/api/org/:orgId/*</code> إلّا المالك أو صاحب صف <code>org_members</code> في	<code>orgStaff()</code> في <code>routes/org.ts</code>
استجابة الفوج إسقاط — حقول جاهزية فقط، ولا صفّ طالب	<code>cohortRow()</code> في <code>org-core.ts</code>
المقعد الذي <code>consent = false</code> يعيد الحالة فقط، مع تفريغ الجاهزية	<code>cohortRow()</code> — صرقة، فهذا اختبار وحدة
لا يضبط الموافقة <code>consent</code> إلّا الطالب نفسه	مسار تحت <code>routes/account.ts</code> ، ميوّب <code>requireUser</code>
لا يكتب <code>org_assignments</code> إلّا مدّرب/مشرف تلك المنظمة	<code>routes/org.ts</code>
إنشاء المقاعد وإزالتها من جانب الخادم فقط؛ ولا مسار كتابة للعميل	صحيح أصلًا — <code>provision-seats</code> ميوّب للمالك

ولن نضيف أمن الصفوف (RLS) في Postgres. فـRLS يحمي قاعدة بيانات من عميل يتصل بها مباشرة؛ ولا يتصل أي عميل بهذه القاعدة. وإضافته طقس يخفي الحدّ الحقيقي، وهو سطح المسارات. أمّا الميزة الوحيدة التي تفقدها هذه المواصفة مما كان في Firestore — ملف سياسة تعريفي قابل للتدقيق باستقلال — فيحدّل محلّها دوال سياسة صرقة في `*-core.ts` مع اختبارات مسارات، وهو النمط الذي تستعمله كلها أصلًا.

6. تدقّق التسجيل والموافقة

1. إنشاء المنظمة — بواسطة Fly GACA عند توقيع العقد، عبر `server/scripts/grant-org.mjs` (يدعم `--dry-run`)، لا خدمة ذاتية. تضبط `name` `owner_user_id` `g` `seat_limit`.

2. يضيف المالك المدرّبين — بالبريد الإلكتروني؛ يحلّ مسار جديد العنوان إلى `users.id` ويُدرج صفًا في

`org_members` بـ `role = 'instructor'`.

3. تسجيل الطلاب — يرسل المالك بريد الطلاب الإلكتروني إلى `POST /api/org/:orgId/provision-seats`

(منشور)، فيكتب صفوف `org_seats` بـ `status = 'invited'` و `consent = false` وقيمة `expires_at`

محدودة. ويُطالب بالمقعد عند أول تسجيل دخول موثّق للطلاب — فتوثيق البريد هو إثبات الملكية، وهي القاعدة نفسها التي تحكم منح الموظفين ونطاقات المدارس.

4. يوافق الطالب — عند تسجيل الدخول التالي يرى الطالب شاشة موافقة: "ستطلع مدرسة الطيران [name] على تقدّمك في المدرسة الأرضية ودرجات اختباراتك التجريبية. ولن تطلع على سجلّك. ويمكنك إلغاء ذلك في أي وقت من الإعدادات" وعند القبول يضبط مساره الخاص `consent = true` و `consent_at = now()`، ويصير المقعد `active`.

5. الإلغاء — الإعدادات ← "مغادرة فوج المدرسة" تضبط `consent = false`. ولا حاجة إلى تفريغ أي شيء في التخزين: فالتقرير يُسقط الجاهزية من المقعد غير الموافق، ومن ثمّ تبقى بيانات الطالب سليمة ويكفّ عن كونها مرئية فحسب.

وتمرّ كل كتابات المقاعد والاستحقاق والعضوية عبر واجهة البرمجة — تمامًا كما يُعالج `entitlements` اليوم، حيث لا يوجد مسار يتيح لعمل أن يكتب خطته بنفسه.

7. خطة واجهة المستخدم

سطح مدرّب جديد بجوار `/business/admin` المنشورة، وكلاهما تحت بوابة `orgStaff` نفسها. تُسجّل المسارات في `src/router.tsx`، محقّلة كسوّلاً، وصفحة لكل مجلّد تحت `src/pages/`، مع نصوص في كلا الملفين `en.json` و `ar.json` (وإلّا أفضل اختبار تكافؤ i18n البناء) وخصائص CSS منطقية كي ينعكس الاتجاه من اليمين إلى اليسار تلقائيًا.

- عرض الفوج — جدول/بطاقات: اسم كل طالب، ونسبة جاهزية الاختبار %، وآخر درجة تجريبية، والدروس المكتملة، وأعلام الحادثة، وآخر نشاط. الفرز + التصفية (مثل "أرني من تأخّر"). وهذا هو "الظلع بلمحة على من هو جاهز ومن يتخلّف" الموعود في `/schools`. (منشور للمالك؛ ويحتاج بوابة المدرّب.)
 - التعمّق لكل طالب — اتجاه جاهزية الطالب، وتاريخ الاختبارات التجريبية، وخريطة دروس المدرسة الأرضية، والحادثة. إضافة إلى عنصر تحكم تعيين مسارات قراءة ← يكتب `org_assignments`؛ ويراها الطالب كمسارات مقترحة.
 - عرض المشرف — المقاعد المستخدمة / الإجمالي، إضافة/إزالة الطلاب والمدرّبين، وملخص جاهزية الفوج (المتوسط، التوزيع، عدد من لم يبدأ بعد). (الإعدادات منشورة.)
 - شاشة موافقة الطالب — تُعرض مرّة واحدة عند التسجيل؛ وتُلخّص مجددًا في `/settings`.
- أعد استخدام إطار صفحة الحساب ومكوّنات البطاقات الموجودة؛ ولا تصنع جديدًا يدويًا.

8. مراحل البناء

المرحلة	النطاق	الحالة	الجهود
A	إسقاط التقدّم من جانب الخادم (<code>studyProgressSync</code> + <code>study_progress</code>)	منجزة	—
A2	تقرير جاهزية الفوج + <code>/business/admin</code> لمالك المنظمة	منجزة	—
B	الموافقة — <code>org_seats.consent</code> ، ومسار موافقة/إلغاء الطالب، والتفريغ في <code>cohortRow()</code>	التالية — بوابة DPIA	S
C	<code>orgStaff()</code> + <code>org_members</code> — دورا المدّرب والمُشرف بما يتجاوز المالك الواحد		M
D	عرض فوج المدّرب + التعقّق لكل طالب		M
E	واجهة إدارة المقاعد + تعيين مسارات القراءة عبر <code>org_assignments</code>		M

المرحلة B صغيرة وحاجزة: فاللوحة حيّة اليوم لمالك واحد، ويجب أن توجد الموافقة قبل أن يكون عليها طلاب مدرسة ثانية.

9. القرارات المفتوحة (أنت)

1. رؤية السجلّ — يرى المدّربون بيانات الدراسة فقط (موصى به، وصحيح حاليًا بحكم البناء)، أم السجلّ أيضًا؟ الخصوصية تقول الدراسة فقط. وتغيير هذا يعني ربطًا جديدًا بجدول `flights` ؛ فلا تفعل.
2. التسجيل — إدخال المالك للبريد الإلكتروني (منشور، تحكّم أكبر) مقابل رمز انضمام (احتكاك أقل، تحكّم أضعف)؟ يمكن دعم كليهما.
3. إنشاء المنظمة — Fly GACA عند العقد عبر `grant-org.mjs` (منشور، وموصى به لمنتج B2B مدفوع) مقابل تسجيل بخدمة ذاتية.
4. الموافقة الافتراضية للمدرسة التجريبية الموجودة على اللوحة أصلًا — موافقة بأثر رجعي (نراسلهم، ونفّرّع الجاهزية حتى يقبلوا) أم استثناء بحكم الأقدمية؟ يوصى بالموافقة بأثر رجعي؛ فهي الإجابة القابلة للدفاع عنها في تقييم DPIA.
5. DPIA — يجب أن تكون مشاركة البيانات العابرة بين المستخدمين ضمن نطاق تقييم أثر حماية البيانات بموجب PDPL. وهذه بوابة إطلاق؛ ولا يمكن توسيع لوحة المدّرب حتى تُغطّى.

10. المخاطر

- فجوة الموافقة. التقرير الحيّ بلا فحص موافقة (35). وهو الإصلاح الأعلى أولويّة؛ فهو مخاطرة امتثال، لا مجرد نقص ميزة.
- إسقاط مسرّب. انتقلت المخاطرة من "خطأ في قاعدة" إلى "مسار يعيد عمودًا زائدًا". التخفيفات: أبقى الإسقاط في `org-core.ts` حيث هو صرف ومُختبَر وحدويًا؛ وأكّد شكل الاستجابة بدقّة في اختبار المسار؛ ولا تستخدم `SELECT *` أبدًا في استجابة فوج.

- PDPL. الرؤية العابرة بين المستخدمين هي بالضبط ما يدققه تقييم أثر حماية البيانات (DPIA) — لا توسّع قبل أن تُغطّى.
- الملخصات المتقدمة. `studyProgressSync` للرفع فقط وببذل أفضل جهد: فمن دون اتصال، أو بلا واجهة برمجة مضبوطة، لا يُرفع شيء ويرى المدرب بيانات قديمة. التخفيف: يحمل التقرير أصلًا `progressUpdatedAt` — أظهره في واجهة المستخدم بحيث يكون التقادم مرئيًا لا صامتًا.
- تضخم النطاق. قاوم المراسلة والتقييم والحضور والفوترة. الإصدار الأول = رؤية جاهزية الفوج. وذلك وحده يبيع المقاعد.